Unit 8: turn code into evidence

CONCEPT & REASONING

A good project answers a specific question, produces a result that can be checked and explains what the result means.

APPLICATION & TAKEAWAY

PYTHON + R + TESTS + CHART + PRESENTATION

What you will produce

CONCEPT & REASONING

One dataset connects two short scripts. The final deliverables are runnable Python and R, a test report and five evidence-focused presentation slides.

APPLICATION & TAKEAWAY

5 days · two languages · five tests · one clear message

The Campus Week Insights question

CONCEPT & REASONING

How much was spent across five campus days? What was the daily average, the highest-spend day, the study-hour total and the allowance status?

APPLICATION & TAKEAWAY

GOAL: explain a small fictional week, not predict future costs

What is in scope?

CONCEPT & REASONING

Analyse exactly five weekdays with non-negative expenditure and study hours. Fixed values are enough; no online service, database or complex application is needed.

APPLICATION & TAKEAWAY

INPUT: 5 days, 5 expenses, 5 study hours, allowance

One shared set of observations

CONCEPT & REASONING

Use matching lists/vectors for days, expenses and hours. Keep them identical in Python and R so the results can be compared meaningfully.

CODE / APPLIED EXAMPLE

Days: Mon Tue Wed Thu Fri
TZS: 2000 3000 0 4000 1000
Hours: 2 1 3 2 2

Know the expected answer

CONCEPT & REASONING

Before writing code, calculate the example result by hand. That gives you an independent standard for checking both programmes.

APPLICATION & TAKEAWAY

Expense total = TZS 10,000
Daily average = TZS 2,000
Study total = 10 hours

Define inputs, process and outputs

CONCEPT & REASONING

Treat the input as raw observations and the outputs as calculated facts. Validate every value before using it in a decision.

APPLICATION & TAKEAWAY

INPUT → VALIDATE → SUMMARISE → CLASSIFY → VERIFY

Write an algorithm first

CONCEPT & REASONING

A language-independent algorithm helps you separate problem-solving from punctuation and syntax.

APPLICATION & TAKEAWAY

STORE → CHECK → CALCULATE → COMPARE → REPORT

Python lists keep rows aligned

CONCEPT & REASONING

The items at the same index belong to the same day. Python indexes the first item at zero.

CODE / APPLIED EXAMPLE

```
days = ["Mon", "Tue", "Wed", "Thu", "Fri"]
expenses = [2000, 3000, 0, 4000, 1000]
```

Store study hours separately

CONCEPT & REASONING

Amounts in TZS and hours are different units. Do not add them together or use them on a single unlabelled axis.

CODE / APPLIED EXAMPLE

```
study_hours = [2, 1, 3, 2, 2]
allowance = 15000
```

Why input validation matters

CONCEPT & REASONING

Without checks, a negative expense or a mismatched list could produce a believable but misleading result.

APPLICATION & TAKEAWAY

VALID: 5 finite, non-negative expenses and hours

Python checks: length

CONCEPT & REASONING

Check that all three lists contain precisely five entries before you calculate statistics or select an index.

CODE / APPLIED EXAMPLE

```
if len(expenses) != 5:
    raise ValueError("Five expenses required")
```

STUDENT STUDY RESOURCE • BASE R • NO EXTRA PACKAGES

Python checks: finite amounts

CONCEPT & REASONING

Use math.isfinite() and a non-negative check. The starter leaves room for students to implement complete validation.

CODE / APPLIED EXAMPLE

```
import math
valid = all(math.isfinite(x) and x >= 0
            for x in expenses)
```

Build small Python functions

CONCEPT & REASONING

Give each part of the logic a clear name. Reusable functions simplify testing and make the main programme easier to understand.

CODE / APPLIED EXAMPLE

```
def total_spending(values):
    return sum(values)
```

Sum expenses once

CONCEPT & REASONING

Adding five observations gives the weekly total. Verify the expected total by independent arithmetic.

CODE / APPLIED EXAMPLE

```
total = sum(expenses)
# 2000 + 3000 + 0 + 4000 + 1000 = 10000
```

Find the daily average

CONCEPT & REASONING

Divide by the number of valid days. A zero-expense day still counts as one observed day.

CODE / APPLIED EXAMPLE

```
average = total / len(expenses)
# 10000 / 5 = 2000 TZS per day
```

Identify the peak day

CONCEPT & REASONING

A maximum tells you a value; use its list index to attach the correct day. Tied maxima should be described accurately.

CODE / APPLIED EXAMPLE

```
top = max(expenses)
first_day = days[expenses.index(top)]
```

Count study hours

CONCEPT & REASONING

Summing the hours gives a second informative measure. Keep the output label and unit distinct from spending.

CODE / APPLIED EXAMPLE

```
total_hours = sum(study_hours)
# 10 hours
```

Budget decision: three states

CONCEPT & REASONING

Distinguish spending above the allowance, exactly equal to it and below it. Equality needs an explicit test.

CODE / APPLIED EXAMPLE

```
if total > allowance:
    status = "Over budget"
elif total == allowance:
    status = "Exactly on budget"
else:
    status = "Within budget"
```

Why equality is a boundary test

CONCEPT & REASONING

A programme can look correct on ordinary inputs but fail when the expense total equals the allowance. Test the edge explicitly.

APPLICATION & TAKEAWAY

total 15000; allowance 15000
EXPECTED: Exactly on budget

Label all printed values

CONCEPT & REASONING

Use descriptive result labels and TZS or hour units. A reader should interpret output without opening the code.

CODE / APPLIED EXAMPLE

```
print(f"Total: TZS {total:,.2f}")
print(f"Study time: {total_hours:g} hours")
```

Use the shared example CSV

CONCEPT & REASONING

CSV stores tabular values in plain text. The provided file is optional for the core tasks; it is a convenient reference for reusing identical data.

CODE / APPLIED EXAMPLE

```
day,expense_tzs,study_hours
Mon,2000,2
Tue,3000,1
Wed,0,3
```

R vectors hold the same data

CONCEPT & REASONING

Use c() and the conventional <- operator. R vectors begin at index one, unlike Python lists.

CODE / APPLIED EXAMPLE

```
expenses <- c(2000, 3000, 0, 4000, 1000)
study_hours <- c(2, 1, 3, 2, 2)
```

R indexing begins at one

CONCEPT & REASONING

R index four corresponds to Thursday. Using index three because of Python habits will select Wednesday instead.

CODE / APPLIED EXAMPLE

```
days[4]  # "Thu"
days[1]  # "Mon"
```

Create a data frame in base R

CONCEPT & REASONING

A data frame arranges named, equal-length columns into rows. It is useful for computing and plotting the shared dataset.

CODE / APPLIED EXAMPLE

```
week <- data.frame(day=days,
 expense=expenses, hours=study_hours)
```

R total and average

CONCEPT & REASONING

R has built-in summary functions. With the same five numeric values, they should agree with Python and hand calculation.

CODE / APPLIED EXAMPLE

```
sum(week$expense)    # 10000
mean(week$expense)   # 2000
```

R: first maximum position

CONCEPT & REASONING

Use which.max() to obtain the first maximum position. Index day labels with that position to report a day name.

CODE / APPLIED EXAMPLE

```
first_peak <-
week$day[which.max(week$expense)]
# "Thu"
```

R: verify numeric input

CONCEPT & REASONING

Make sure values are numeric, finite, and non-negative. Do not silently turn missing or negative input into a valid zero.

CODE / APPLIED EXAMPLE

```
stopifnot(length(expenses)==5)
stopifnot(all(is.finite(expenses)))
stopifnot(all(expenses>=0))
```

Make a labelled bar chart

CONCEPT & REASONING

Each bar represents one daily expense. Include the day labels, title and currency unit for a readable result.

CODE / APPLIED EXAMPLE

```
barplot(week$expense,
names.arg=week$day,
 main="Campus week expenses",
ylab="Amount (TZS)")
```

Describe the figure

CONCEPT & REASONING

The supplied example shows Thursday as the largest expense (TZS 4,000) and Wednesday as zero. The chart describes observations rather than explaining causes.

APPLICATION & TAKEAWAY

OBSERVATION ≠ CAUSAL EXPLANATION

Compare language outputs

CONCEPT & REASONING

Record total, mean, top day, study hours and budget status from both scripts. Agreement is a consistency check, not automatic proof.

APPLICATION & TAKEAWAY

Python total: 10000 TZS
R total: 10000 TZS
RESULT: match

Test T01: normal week

CONCEPT & REASONING

Confirm the standard worked example before changing inputs. Calculate its expected results independently.

APPLICATION & TAKEAWAY

Total 10000; mean 2000; top Thu
Hours 10; within budget

Test T02: all zero

CONCEPT & REASONING

All zero values should be accepted. With zero allowance, spending is exactly on budget. The first tied maximum is Monday.

APPLICATION & TAKEAWAY

Expenses 0,0,0,0,0; allowance 0
Total 0; mean 0; first peak Mon

Test T03: exact allowance

CONCEPT & REASONING

Set the total expense to the allowance to catch comparison mistakes. Keep study hours visible in the expected result.

APPLICATION & TAKEAWAY

1k+2k+3k+4k+5k = 15k

Allowance 15k → Exactly on budget

Test T04: over allowance

CONCEPT & REASONING

The example totals TZS 22,000 against a TZS 20,000 allowance. A correct programme reports a shortfall of TZS 2,000.

APPLICATION & TAKEAWAY

Total 22000; mean 4400; top Wed
Allowance 20000 → Over budget

Test T05: invalid negative

CONCEPT & REASONING

An expense of −100 violates this project's rule. Validating first prevents a misleading sum and budget status.

APPLICATION & TAKEAWAY

[2000, -100, 1000, 3000, 500]
EXPECTED: rejected; no valid summary



Expected versus actual

CONCEPT & REASONING

A meaningful test report records what should happen and what happened, then states PASS or FAIL. Note and fix disagreements.

APPLICATION & TAKEAWAY

TEST ID | EXPECTED | ACTUAL | RESULT

What to do after a failure

CONCEPT & REASONING

Check the data and lengths, isolate the failing function, correct the smallest cause, and rerun all earlier tests.

APPLICATION & TAKEAWAY

OBSERVE → LOCATE → CORRECT → RETEST

Explain privacy and limitations

CONCEPT & REASONING

The sample is fictional. Never publish real private spending records without permission. Five days cannot establish future trends.

APPLICATION & TAKEAWAY

LIMITATION: sample week; no forecasting claim

From a real problem to a manageable project

CONCEPT & REASONING

A project begins with a problem experienced by a defined user or group. A problem statement identifies the situation, the difficulty and the intended benefit. Requirements describe what the solution must do and what counts as success. A small project can be useful without advanced AI or a graphical interface.

EXAMPLE & RESULT

```
Example: Rainfall log
Problem: daily rainfall summaries are calculated
manually.
Input: five daily measurements in mm.
Output: total, average, wet-day count and chart.
Success: results match independent tests.
```

A narrow, testable scope is achievable using lists, functions and R data frames.

A project lifecycle and evidence

CONCEPT & REASONING

The lifecycle connects problem definition, requirements, data description, algorithm, implementation, testing, documentation and presentation. Each stage leaves an artefact that supports the next: a clear requirement leads to a test, and a test result supports the final claim. Feedback can lead to a revised requirement or implementation.

EXAMPLE & RESULT

```
Problem → Requirements → Data & algorithm
→ Implementation → Tests → Report
→ Presentation → Feedback & revision
```

The sequence is iterative: a failing test may reveal an incorrect algorithm or ambiguous requirement.

Independent project themes

CONCEPT & REASONING

The Campus Week Insights project remains the required shared assessment. The same concepts transfer to optional practice themes: fictional student marks (mean and categories), crop harvest (totals by location), rainfall (wet days and weekly totals), and study habits (hours against target). Each theme needs a data dictionary, valid ranges, expected outputs and equivalent Python/R calculations.

EXAMPLE & RESULT

```
Rainfall: mm, non-negative, five days
Harvest: kg, non-negative, five plots
Marks: 0–100, four fictional students
Study: hours, 0–24 per day
```

These themes are optional extensions; they do not replace the published assignment or its marking scheme.

Data quality and an explicit data dictionary

CONCEPT & REASONING

A data dictionary describes each variable, its type, units, valid range and missing-value policy. Five rows of fictional data avoid unnecessary collection of personal information. Missing rainfall is unknown, not zero; a numeric summary needs a declared policy for excluding missing observations or rejecting incomplete records.

EXAMPLE & RESULT

```
Column        Type       Units
day           text       weekday
rainfall      numeric    mm
Valid range: rainfall >= 0
Missing: record as NA, report count
Example: Mon, 0; Tue, 5; Wed, NA
```

Zero is a known no-rain observation. NA is missing. A mean of observed values uses the number of available observations, not all rows.

Reporting ties and interpreting results

CONCEPT & REASONING

The Campus Week example reports the first maximum day when values tie. This is a valid stated rule, but it does not identify every maximum. A report requiring all tied days uses a filter instead. Requirements determine which result is appropriate.

EXAMPLE & RESULT

```
days = ["Mon", "Tue", "Wed"]
values = [5, 8, 8]
peak = max(values)
for i in range(len(values)):
    if values[i] == peak:
        print(days[i])
```

Tue and Wed. The original required project retains its first-maximum rule; this example illustrates a different requirement.

Evidence-based presentation and responsible assistance

CONCEPT & REASONING

A project presentation explains the need, data, algorithm, results and limitations. A chart without its units and context does not establish a conclusion. A short demonstration of a known test and a modified input provides evidence that the presenter understands the code. Any AI or external assistance is documented with its purpose and the verification performed.

EXAMPLE & RESULT

```
Claim: five days total 10 mm
Evidence: data + expected total + actual output
Limitation: fictional week, not a forecast
AI assistance: explanation of a loop;
independent test confirmed the result.
```

A strong report distinguishes measured results from assumptions and avoids claiming real-world accuracy from fictional examples.

Write a short report

CONCEPT & REASONING

Include a problem statement, dataset, algorithm, Python/R implementation, key results, test table, limitations and assistance declaration.

APPLICATION & TAKEAWAY

REPORT: Problem → Data → Method → Results → Tests

Presentation slide 1: purpose

CONCEPT & REASONING

Explain the practical question and the intended user. Keep the project title specific and the scope modest.

APPLICATION & TAKEAWAY

QUESTION: What did one fictional campus week show?

Presentation slide 2: data and method

CONCEPT & REASONING

State the five observations and the processing steps. A simple labelled flow or short table is clearer than a wall of code.

APPLICATION & TAKEAWAY

INPUT → VALIDATION → ANALYSIS

Presentation slide 3: two implementations

CONCEPT & REASONING

Mention the different strengths of each solution: Python calculations/functions; R tabular data, independent checks and a chart.

APPLICATION & TAKEAWAY

PYTHON: calculate + validate
R: verify + visualise

Presentation slide 4: results and tests

CONCEPT & REASONING

Show the main totals, the labelled R bar chart and evidence for required test scenarios. Avoid claiming that a chart establishes causation.

APPLICATION & TAKEAWAY

Total TZS 10000 · Average TZS 2000
Highest Thu TZS 4000 · 5 tests

Presentation slide 5: learning and limits

CONCEPT & REASONING

Close with what works, what you tested and one limitation. Be able to run the code and change a value when asked.

APPLICATION & TAKEAWAY

LIMITS: fictional five-day data
NEXT: longer datasets are a possible extension

Final submission and next steps

CONCEPT & REASONING

Submit runnable Python and R scripts, report with five tests and comparison table, and five-slide presentation. Keep your own backups.

APPLICATION & TAKEAWAY

SCRIPTS + TEST REPORT + CHART + FIVE SLIDES